

CSC 330 Programming Languages

Program #3

Scheme

Note:

As mentioned in the notes, since there are multiple versions of Scheme, it is possible that you write perfectly valid code that I can't compile and run. So, to get everyone on the same page, you must use the racket language. Your language should be set by specifying this at the top of your file:

```
#lang racket
```

Directions:

For this assignment you will be creating a simulated password cracker. You will give it a list of sha1 hashes that come from passwords, and it will brute force all permutations of a given alphabet in an attempt to find the password that generated each hash. To make things simple, use the following alphabet for your passwords:

```
(define alphabet '("a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n"
                  "o" "p" "q" "r" "s" "t" "u" "v" "w" "x" "y" "z"))
```

Take the first hash from the list and try all single letter passwords in order ("a", "b", "c", ...), hashing each one and comparing to see if any match the given hash. After exhausting all single letters, try all permutations of 2 letters ("aa", "ab", "ac", ..., "ba", "bb", "bc", ...), then 3 letters, and so on until the password is found. None of these passwords are more than 6 letters so if you want, you can make it stop after exhausting all 6 letter permutations. You could also just press ctrl+c in the terminal to end the running program after too much time has passed. Once a password is found, move on to the next hash. Here are your given hashes:

```
(define hashes '("fd1286353570c5703799ba76999323b7c7447b06"
                 "66b27417d37e024c46526c2f6d358a754fc552f3"
                 "39ccb32d95edfdbcd882f2b01809724ec640ea16"
                 "8abf15bef376e0e21f1f9e9c3d74483d5018f3d5"
                 "163e65be076bbea20ab8275969700373a6179a3c"))
```

Hashing:

Scheme has a built in function for generating a sha1 hash. If you aren't familiar with what this is, essentially it takes a string of any length and returns a fixed length hexadecimal number. When you create a password for a website, the password you give is not saved in some database. It is hashed first. This turns your password into a series of meaningless characters that cannot be reversed to reveal what your true password. So, if a hacker gets access to the database, they will not receive your password, instead they will receive this hash. The only way to find what your actual password is, is to try a bunch of different possible passwords, hash each one, and compare the hash to see if that's the password that generated it.

To get Scheme to generate a sha1 hash, you will need to include the proper library. Put the following at the top of your file right under the #lang racket part:

```
(require file/sha1)
```

Now you can hash a string by calling the following function:

```
(sha1 (open-input-string "possible_password"))
```

Timing:

You will need to time how long each hash takes to crack. To time a function call, simply use the time function. For example, to time how long it takes to call a custom function named "crack", use the following:

```
(time (crack hash))
```

This will add some text to the output that shows the amount of time taken (in milliseconds).

Functional style:

Scheme is a functional language. To be proficient in Scheme, you need to think about logic from a functional paradigm perspective. This means recursion is your only source of repetition and any time you have a collection of values, they need to be in a list. Using functions like fold, map, and for-each is recommended for processing items in a list. This programming assignment is going to be difficult not only because you are using a new language, but because the way you think about problem solving needs to be different as well. It is recommended to allocate plenty of time to work on this project. Remember that you may be selected to come to my office and explain your code. So, any use of code found online or generated is strongly discouraged as you will have to explain (in detail) how that code works.

Sample output:

Your output should show both the hash and the found password, along with the time it took (the "time" function explained above will add this part for you). Your output should look like the following (with "??" replaced with the actual found password in quotes):

```
fd1286353570c5703799ba76999323b7c7447b06 is "???"  cpu time: 0 real time: 0 gc time: 0
66b27417d37e024c46526c2f6d358a754fc552f3 is "???"  cpu time: 15 real time: 22 gc time: 0
39ccb32d95edfdbc882f2b01809724ec640ea16 is "???"  cpu time: 62 real time: 149 gc time: 0
8abf15bef376e0e21f1f9e9c3d74483d5018f3d5 is "???"  cpu time: 640 real time: 657 gc time: 15
163e65be076bbea20ab8275969700373a6179a3c is "???"  cpu time: 34468 real time: 34928 gc time: 46
```

Your times might be slightly different but should be in the ballpark of the times above.

For submission:

Submit only your .scm file. Name the file "crack.scm" and submit on Canvas.

Rubric:

#	ITEM	POINTS
1	Correct passwords found	20
2	Timing is accurate	20
3	Output matches	20
	TOTAL	60

Penalties

#	PENALTIES	POINTS
1	Doesn't compile	-50%
2	Doesn't execute once compiled (i.e. it crashes)	-25%
3	Late up to 1 day	-25%
4	Late up to 2 days	-50%
5	Late after 2 days	-100%